

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA



Informe Técnico: Análisis, Diseño y Codificación del Sistema Lab-Inventario

Aplicación de los principios SOLID, arquitectura en capas y patrones de diseño en Java con
persistencia SQLite

Mateo Sebastian Jimenez Esparza

Servio David Valverde Lopez

Programación Orientada a Objetos

Docente: Wayner Xavier Bustamante

Loja, Ecuador

24 de julio de 2026

Introducción

El presente informe documenta el proceso de análisis, diseño y codificación del sistema Lab-Inventario, una aplicación de escritorio desarrollada en Java cuyo propósito es gestionar los activos de un laboratorio informático —equipos de hardware, periféricos y licencias de software— incluyendo su registro, consulta, actualización, eliminación y el cálculo del costo de mantenimiento asociado a cada tipo de activo.

El desarrollo del sistema se sustenta en tres pilares técnicos: la aplicación consciente de los cinco principios SOLID, una arquitectura organizada en capas (modelo, repositorio, servicio, controlador y vista de consola) inspirada en el patrón Modelo-Vista-Controlador, y la persistencia de datos mediante SQLite a través de JDBC. Estas decisiones de diseño buscan que el sistema sea mantenible, extensible y fácil de probar, características deseables en cualquier software de mediana complejidad.

A continuación, se presenta el análisis que justifica la aplicación de cada principio SOLID en el código fuente, el diseño detallado del sistema a través de su diagrama de clases UML, y finalmente la codificación, donde se explican los patrones de diseño empleados junto con los fragmentos de código más relevantes.

1. Análisis: Justificación de la Aplicación de Principios SOLID

Los principios SOLID —Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion— constituyen un conjunto de buenas prácticas para el diseño orientado a objetos que favorecen la mantenibilidad y la extensibilidad del software (Martin, 2000). En el sistema Lab-Inventario, cada uno de estos principios se aplica de manera deliberada y verificable en el código, tal como se justifica a continuación.

1.1 Principio de Responsabilidad Única (SRP)

El principio de responsabilidad única establece que una clase debe tener una, y solo una, razón para cambiar. En el proyecto, este principio se evidencia en la separación estricta entre capas: la clase abstracta `Activo` y sus subclases (`Hardware`, `Periferico`, `Licencia`) solo contienen datos y la lógica de negocio propia del activo (por ejemplo, el cálculo de su costo de mantenimiento); no conocen nada sobre SQL ni sobre cómo se persisten los datos. Esta responsabilidad de persistencia recae exclusivamente en `ActivoRepositorySQLite`. De igual forma, `ActivoService` concentra la lógica de negocio orientada al caso de uso (registrar, buscar, listar, actualizar, eliminar y calcular costos), `ActivoController` expone esa lógica a la capa de presentación, y `ConsolaView` se limita a la interacción con el usuario por consola. Cada clase tiene, por tanto, un único motivo para cambiar.

1.2 Principio de Abierto/Cerrado (OCP)

El principio de abierto/cerrado indica que las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación. La jerarquía de activos se diseñó pensando en este principio: el método abstracto `calcularCostoMantenimiento()` se declara en `Activo` y cada subclase lo implementa según su propia fórmula de costos (desgaste por antigüedad en `Hardware`, recargo por proximidad de expiración en `Licencia`, porcentaje fijo en `Periferico`). Si en el futuro se necesitara agregar un nuevo tipo de activo —por ejemplo, `Mobiliario`—, bastaría con crear una nueva subclase de `Activo` que implemente los métodos abstractos, sin modificar `ActivoService`, `ReporteService` ni ningún código cliente existente que dependa de la abstracción.

1.3 Principio de Sustitución de Liskov (LSP)

El principio de sustitución de Liskov exige que los objetos de una clase derivada puedan reemplazar a los de su clase base sin alterar la corrección del programa. En el sistema, `Hardware`,

Periferico y Licencia son completamente sustituibles por su tipo base Activo en cualquier contexto: la lista devuelta por `ActivoRepository.listarTodos()` es `List<Activo>`, y tanto `ActivoService.calcularCostoMantenimientoTotal()` como `ReporteService` recorren esa lista invocando `calcularCostoMantenimiento()` de forma polimórfica, sin necesidad de conocer ni verificar el tipo concreto de cada elemento. Ninguna subclase restringe precondiciones ni debilita la garantía del contrato definido por la clase abstracta, por lo que el comportamiento se mantiene consistente y predecible.

1.4 Principio de Segregación de Interfaces (ISP)

El principio de segregación de interfaces recomienda que los clientes no deban depender de métodos que no utilizan. En lugar de definir una única interfaz extensa que mezcle operaciones CRUD con operaciones de reportería, el proyecto separa explícitamente `ActivoRepository` (guardar, buscarPorId, listarTodos, actualizar, eliminar) de `Reportable` (filtrarPorTipo, costoTotalMantenimiento). `ActivoService` solo depende de `ActivoRepository`, mientras que `ReporteService` solo depende de `Reportable`, aunque ambas interfaces sean implementadas por la misma clase concreta `ActivoRepositorySQLite`. De esta manera, ningún cliente se ve forzado a depender de métodos de reportería que no necesita, o viceversa.

1.5 Principio de Inversión de Dependencias (DIP)

El principio de inversión de dependencias establece que los módulos de alto nivel no deben depender de módulos de bajo nivel, sino que ambos deben depender de abstracciones. En `Lab-Inventario`, `ActivoService` y `ReporteService` dependen únicamente de las interfaces `ActivoRepository` y `Reportable`, nunca de la clase concreta `ActivoRepositorySQLite`. La única sección del sistema donde se instancia directamente la implementación concreta es `Main.java`, que

actúa como punto de composición o ensamblado (composition root) e inyecta esa dependencia a través de los constructores de los servicios. Esta decisión permite sustituir la persistencia en SQLite por otra implementación (por ejemplo, una en memoria para pruebas unitarias) sin modificar una sola línea de la capa de servicio.

2. Diseño: Diagrama UML Detallado

El diseño del sistema se modeló mediante un diagrama de clases UML que representa la totalidad de las capas del proyecto: modelo (model), repositorio (repository), servicio (service), controlador (controller) y el punto de entrada de la aplicación (pkg2dobim.Main). La Figura 1 muestra el diagrama completo, elaborado en la herramienta Dia.

2.1 Descripción de las clases del paquete model

Activo es una clase abstracta que actúa como raíz de la jerarquía de activos. Declara los atributos protegidos comunes `id`, `nombre`, `fechaAdquisicion`, `costoBase` y `estado`, junto con sus respectivos métodos de acceso (getters/setters), el método protegido `antiguedadEnAnios()` y dos métodos abstractos que obligan a cada subclase a definir su propio comportamiento: `calcularCostoMantenimiento()`, que devuelve un valor `double`, y `getTipo()`, que devuelve un valor del enumerado `TipoActivo`. También sobrescribe `toString()`, `equals()` y `hashCode()` para un comportamiento coherente basado en el identificador del activo.

Hardware, Periferico y Licencia son subclases concretas que heredan de Activo mediante una relación de generalización (representada en el diagrama con flechas de punta triangular vacía). Hardware añade los atributos vidaUtilAnios y marca, y calcula su mantenimiento aplicando un cargo fijo del 10% del costo base más un recargo variable de hasta el 15% en función del desgaste por antigüedad. Periferico añade tipoConexion y calcula un mantenimiento simple del 3% del costo base. Licencia añade fechaExpiracion y numeroLicencias, y calcula su mantenimiento aplicando un recargo del 20% del costo base, incrementado en un 20% adicional si la licencia está próxima a expirar (30 días o menos).

TipoActivo es un enumerado con los valores HARDWARE, PERIFERICO y LICENCIA. Se utiliza como discriminador de tipo tanto para la persistencia en la base de datos como para la generación de reportes, evitando el uso de instanceof en las capas superiores.

2.2 Descripción de las clases del paquete repository

ActivoRepository es una interfaz que define el contrato CRUD del sistema: guardar(Activo), buscarPorId(int), listarTodos(), actualizar(Activo) y eliminar(int). Reportable es una segunda interfaz, independiente de la anterior, que define las operaciones de reportaría: filtrarPorTipo(TipoActivo) y costoTotalMantenimiento(). ActivoRepositorySQLite es la única clase concreta del sistema que implementa ambas interfaces (relación de realización, representada en el diagrama con línea discontinua y punta triangular vacía) utilizando JDBC sobre una base de datos SQLite. Internamente contiene los métodos privados enlazarParametros() y mapearActivo(), que traducen entre objetos Activo y filas de la tabla activos. ConexionSQLite es una clase de utilidad con constructor privado y un único método estático obtenerConexion(), responsable de cargar el driver JDBC y abrir la conexión. DatabaseInitializer, también con constructor privado,

expone el método estático `inicializar(Connection)` que crea la tabla `activos` si no existe. `RepositorioException` es una excepción de tipo `RuntimeException`, utilizada de forma consistente para envolver y propagar los errores de acceso a datos.

2.3 Descripción de las clases de los paquetes `service` y `controller`

`ActivoService` depende únicamente de la interfaz `ActivoRepository` (relación de asociación/dependencia dirigida hacia la abstracción) y expone los casos de uso de negocio: `registrarActivo`, `obtenerActivo`, `listarActivos`, `actualizarActivo`, `eliminarActivo`, `calcularCostoMantenimientoTotal` y `calcularCostoMantenimientoPorActivo`. `ReporteService` depende únicamente de la interfaz `Reportable` y ofrece `imprimirCostoMantenimientoPorTipo()` e `imprimirCantidadActivosPorTipo()`. `ActivoController` depende de `ActivoService` y traduce las operaciones de negocio en operaciones invocables desde la capa de presentación (crear, buscar, listar, actualizar, eliminar). `MantenimientoController` depende tanto de `ActivoService` como de `ReporteService`, orquestando los cálculos de mantenimiento y la generación de reportes. Finalmente, `ConsolaView` depende de ambos controladores y constituye la capa de presentación basada en texto, con un bucle de menú que lee la entrada del usuario mediante `Scanner` y delega cada operación al controlador correspondiente.

La clase `Main` (paquete `pkg2dobim`) actúa como punto de composición: obtiene la conexión `JDBC`, inicializa el esquema, instancia `ActivoRepositorySQLite` y, a partir de ahí, construye en cadena los servicios, controladores y la vista de consola, inyectando cada dependencia por constructor. Es el único lugar del sistema donde se referencia explícitamente la implementación concreta del repositorio, lo cual es coherente con la inversión de dependencias descrita en la sección de análisis.

3. Codificación: Patrones Aplicados y Fragmentos Relevantes

Además de los principios SOLID, en la codificación del sistema Lab-Inventario se reconocen varios patrones y prácticas de diseño de software ampliamente documentados en la literatura (Gamma, Helm, Johnson y Vlissides, 1994), aplicados sobre la estructura mostrada en la Figura 1. A continuación, se explica cada uno junto con los fragmentos de código que lo evidencian.

3.1 Arquitectura en capas (estilo MVC)

El proyecto organiza su código en cinco paquetes con responsabilidades claramente delimitadas: model (entidades del dominio), repository (acceso a datos), service (lógica de negocio), controller (orquestración de casos de uso) y la vista de consola dentro de service.ConsolaView. Esta separación, visible en el diagrama de la Figura 1 a través de la disposición por paquetes y las dependencias unidireccionales entre ellos (vista → controlador → servicio → repositorio → base de datos), reduce el acoplamiento y permite modificar una capa sin afectar a las demás.

3.2 Patrón Repository

El patrón Repository encapsula el acceso a la base de datos detrás de una interfaz orientada al dominio, ocultando los detalles de JDBC y SQL al resto de la aplicación. La interfaz ActivoRepository define el contrato, y ActivoRepositorySQLite lo implementa concretamente:

```
public interface ActivoRepository {  
    void guardar(Activo activo);  
    Optional<Activo> buscarPorId(int id);  
    List<Activo> listarTodos();  
    void actualizar(Activo activo);  
    void eliminar(int id);  
}
```

Gracias a este patrón, `ActivoService` nunca invoca directamente sentencias SQL; simplemente colabora con la abstracción `ActivoRepository`, lo que facilita sustituir la fuente de datos o crear implementaciones de prueba (mocks) para pruebas unitarias.

3.3 Inyección de dependencias por constructor (Composition Root)

En lugar de que cada clase construya sus propias dependencias, estas se reciben por constructor y se ensamblan una única vez en Main, siguiendo el patrón de composition root:

```
ActivoRepositorySQLite repositorioConcreto = new ActivoRepositorySQLite(conexion);
ActivoRepository activoRepository = repositorioConcreto;

ActivoService activoService = new ActivoService(activoRepository);
ReporteService reporteService = new ReporteService(repositorioConcreto);

ActivoController activoController = new ActivoController(activoService);
MantenimientoController mantenimientoController =
    new MantenimientoController(activoService, reporteService);
```

Este fragmento evidencia directamente la inversión de dependencias descrita en la sección 1.5: `activoRepository` se declara con el tipo de la interfaz, aunque el objeto real sea `repositorioConcreto`.

3.4 Polimorfismo como estrategia de cálculo (equivalente a Strategy)

Cada subclase de `Activo` encapsula su propio algoritmo de cálculo de mantenimiento, lo que en la práctica cumple el mismo propósito que el patrón Strategy: variar un algoritmo de forma independiente del cliente que lo utiliza.

```
// Hardware.java
@Override
public double calcularCostoMantenimiento() {
    int antiguedad = antiguedadEnAnios();
    double factorDesgaste = vidaUtilAnios > 0
        ? Math.min(1.0, (double) antiguedad / vidaUtilAnios)
        : 0.5;
    return costoBase * 0.10 + costoBase * 0.15 * factorDesgaste;
}
```

```
// Licencia.java
@Override
public double calcularCostoMantenimiento() {
    long diasParaExpirar = fechaExpiracion != null
        ? ChronoUnit.DAYS.between(LocalDate.now(), fechaExpiracion) : 365;
    double recargoUrgencia = diasParaExpirar <= 30 ? 1.20 : 1.0;
    int licencias = Math.max(1, numeroLicencias);
    return (costoBase / licencias) * 0.20 * licencias * recargoUrgencia;
}
```

El código cliente (`ActivoService.calcularCostoMantenimientoTotal()`) recorre la lista de activos sin conocer qué fórmula se ejecuta en cada caso:

```
public double calcularCostoMantenimientoTotal() throws RepositorioException {
    return activoRepository.listarTodos().stream()
        .mapToDouble(Activo::calcularCostoMantenimiento)
        .sum();
}
```

3.5 Utilidad estática con constructor privado (Static Factory / Singleton de acceso)

`ConexionSQLite` y `DatabaseInitializer` aplican el patrón de clase de utilidad: constructor privado para impedir la instanciación y un único método estático de acceso, garantizando un punto centralizado y consistente para obtener la conexión JDBC e inicializar el esquema:

```
public class ConexionSQLite {
    private static final String URL = "jdbc:sqlite:2doBim.db";
    private ConexionSQLite() {}

    public static Connection obtenerConexion() {
        try {
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
            throw new RepositorioException("No se encontro el driver JDBC de SQLite",
e);
        }
        try {
            return DriverManager.getConnection(URL);
        } catch (SQLException e) {
            throw new RepositorioException("No se pudo conectar a la base de datos
SQLite", e);
        }
    }
}
```

3.6 Manejo centralizado de excepciones

Todas las operaciones de acceso a datos que pueden fallar propagan una única excepción de dominio, `RepositorioException`, en lugar de exponer `SQLException` directamente a las capas superiores. Esto desacopla la capa de servicio de los detalles de JDBC y permite un manejo de errores uniforme en la vista de consola:

```
public class RepositorioException extends RuntimeException {  
    public RepositorioException(String mensaje, Throwable causa) { super(mensaje,  
causa); }  
    public RepositorioException(String mensaje) { super(mensaje); }  
}
```

Como se observa en la Figura 1, `RepositorioException` se relaciona con `ActivoRepositorySQLite` y es capturada en `ConsolaView.iniciar()`, que envuelve cada operación del menú en un bloque try/catch para informar el error al usuario sin interrumpir la ejecución del programa.

Conclusiones

El sistema Lab-Inventario demuestra la aplicación práctica y coherente de los cinco principios SOLID sobre una arquitectura en capas: cada clase mantiene una responsabilidad única, la jerarquía de activos permite extender el sistema sin modificar código existente, las subclasses de `Activo` son plenamente sustituibles por su tipo base, las interfaces se mantienen pequeñas y específicas, y la lógica de negocio depende exclusivamente de abstracciones en lugar de implementaciones concretas.

El diseño representado en el diagrama UML evidencia una separación clara de responsabilidades entre modelo, repositorio, servicio, controlador y vista, mientras que la codificación pone en práctica patrones reconocidos —Repository, inyección de dependencias por

constructor, polimorfismo como estrategia de cálculo y manejo centralizado de excepciones— que en conjunto favorecen la mantenibilidad, la extensibilidad y la facilidad de prueba del sistema.

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Martin, R. C. (2000). Design principles and design patterns. Object Mentor.
https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf

Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.

Oracle. (2024). The Java tutorials: JDBC database access. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/jdbc/>

SQLite Consortium. (2024). SQLite documentation. <https://www.sqlite.org/docs.html>